Microformat and Message Versioning

Table of Contents

References . . . 1
1. Principles . . . 1
1.1. Microformats . . . 1
1.2. Messages . . . 2
2. Version determination . . . 3
3. Validation . . . 4
3.1. Facets . . . 5
4. Applications . . . 6
5. Code Generators . . . 6
5.1. Generated Code Compatibility . . . 6
6. Distribution of Definitions . . . 8
6.1. Current Position . . . 8
7. Summary . . . 9
7.1. Problems . . . 9
7.2. Solutions . . . 10

References

- [SPARTAN SHIELD Proposal] Versioning: Proposal for SPARTAN SHIELD, Version 0.1, January 2023
- [Principles] AOCO.511 Micro Format and Message Definition Principles, Revision 1.3.1, 8 August 2022
- [Message DSL] AOCO.744 Common Services Messages Domain Specific Language, Revision 1.2, 8 August 2022

1. Principles

It is useful to point out some pertinent principles regarding Microformat and Message versioning. See [Principles] Section 5.3 and Section 6.5.2 for further details.

1.1. Microformats

1.1.1. Microformat Version Compatibility

- A microformat definition is versioned with a major and minor version.
- Realized microformats created using definitions with the same major version are forwards and backwards compatible with each other.
- Realized microformats created using definitions with a different major version are not forwards and backwards compatible and cannot be substituted for one another.
- To ensure compatibility between minor versions:
  - Unrecognised fields should be ignored by any consumer.
  - All fields added or removed must be optional.
  - No value constraints can be changed.

1.1.2. Microformat Definition Issues

A microformat can reference another microformat as an `External` type or `Internal` type in order to create a structured hierarchy of data within the microformat, or as a `Supertype`. `Internal` references are defined in the same file as the main microformat, so do not require any version information, as they are defined within the context of the file version. Both `External` and `Supertype` microformats reference microformats defined in other files, (and potentially other packages), that are independently versioned.

At present there is no way to specify which version of and external or supertype microformat to use. When mentioning where version information is known in the rest of this document, it has been assumed that this issue will be resolved. The proposal in [SPARTAN SHIELD Proposal] of extending the DSL to add a version field to the reference, with a default value of 1 if not present seems a sound one.

1.2. Messages

1.2.1. Message Version Compatibility

- A Message definition is versioned with a major and minor version.
- A message definition specifies the exact version of each microformat that it contains.
- A message definition specifies the exact version of each collection that it uses.
- A message definition specifies the exact version of any inherited message that it uses.
- Realized messages created using definitions with the same major version are forwards and backwards compatible with each other.
- Realized messages created using definitions with a different major version are not forwards and backwards compatible and cannot be substituted for one another.
- To ensure compatibility between minor versions:
  - Unrecognised microformats should be ignored by any consumer.
  - All microformats added or removed must be optional.

- Any microformat versions used must remain on the same major version.
- All collections added or removed must be optional.
- Any collection versions used must remain on the same major version.
- An inherited message cannot be added or removed.
- Any inherited message must remain on the same major version.
- Any inherited message cannot change insertion point path
- An association cardinality can only change from `required` to `oneOrMore` and *vice-versa*, or from `optional` to `zeroOrMore` and *vice-versa*.
- No facets can be changed.
- No inclusion rules can be added or changed. (See Section 3.1, “Facets” below)

1.2.2. Collections

- A collection definition is versioned with a major and minor version
- To ensure compatibility between minor versions:
  - The rules for messages above must be followed.

1.2.3. Message Extensions

- A message extension definition is versioned with a major and minor version.
- A message extension may define extensions to multiple different messages.
- A message extension specifies the major and minor version of all the messages it is extending.
- Realized message extensions created using definitions with the same major version are forwards and backwards compatible with each other.
- A realized message extension is a valid realization of the message it is extending. Note that unrecognised microformats are ignored.
- To ensure compatibility between minor versions:
  - The rules for messages above must be followed.
  - All extended message must remain on the same major version.
  - The set of extended messages must remain the same.
  - Insertion point paths for each extended message must remain the same.

2. Version determination

A realized microformat or message does not by definition expose which version of the definition was used to create it. This information must be inferred from elsewhere.

No microformat message definitions within the Common Services remit contain any way to infer the definition version from the data contained in that microformat in isolation.

All message definitions within the Common Services remit contain a `MsgDistMf` microformat, which details which message definition and version was used to create the message. Although not a requirement of the specification detailed in [Principles], it will be assumed for the purposes of this document that this microformat is present in all valid message definitions.

The presence of this microformat means that the correct message definition version is known when handling a message. In turn, this means that the version of each microformat defined within the message is known. Note, however, that it is not possible to determine the version of any additional microformats present in a realized message that are not present in the message definition.

3. Validation

The forwards and backwards compatibility rules effectively mean that a given realization of a message or microformat should be a valid against all definitions with the same major version number. Any tool validating a message should be able to choose any definition with the same major version number and use that definition to correctly validate a message or microformat. The presence of unrecognised fields or microformats should not cause validation to fail.

Note, however, that the level of validation possible will change depending on whether optional fields or microformats have been added or removed between versions.

For example

- If version 1.1 of a microformat adds an optional field `alpha` onto the 1.0 definition, a validator using the 1.0 definition would not be able to validate this field, as it knows nothing about what values the field can take.
- If version 1.0 of a microformat has an optional field `alpha`, which is removed in the 1.1 definition, a validator using the 1.1 definition would not be able to validate this field, as it knows nothing about what values the field can take.
- If version 1.0 of a message has an optional microformat `BetaMf`, which is removed in version 1.1, a validator using the 1.1 definition would not be able to validate the AlphaMf microformat, as it would not know which version to validate against.
- If version 1.1 of a message adds an optional microformat `BetaMf` compared to 1.0, a validator using the 1.0 definition would not be able to validate the AlphaMf microformat, as it would not know which version to validate against, and might not even have a definition for the microformat available.

A validator will be unable to validate a message or microformat for which it does not have at least one definition with a matching major version.

It is clear from the above that at a minimum, a validator needs

- One definition with a matching major version for each message that it is intended to validate
- One definition with a matching major version for each microformat used by those message definitions

For the maximum level of validation, a validator needs

- All versions of definitions with a matching major version for each message that it is intended to validate.
- Every definition with a matching major and minor version for each microformat used by any of the above message definitions.

So, a validator wishing to validate messages for multiple different IDDs and IDD versions, will need definitions for all messages and microformat versions used within those IDDs.

3.1. Facets

Although not explicitly mentioned in [Principles], a Facet (otherwise known as a Field Constraint) ([Principles] Section 8.6) cannot be altered between minor versions whilst maintaining forwards and backwards compatibility.

The MsgDistMf microformat within each message definition currently has its `type/version` field constrained to a fixed value of the major and minor version number. This has the effect of preventing a message validating against a definition other than for the exact version it was created against, breaking both forward and backwards compatibility for minor versions. Some consideration needs to be given to how to solve this issue.

Validators could be built with special case handling for this field. This breaks domain encapsulation, with validators needing special knowledge about the inner workings of a particular microformat, but it could be argued that this encapsulation is already broken by both this field and the `type/name` field being used to choose the correct definition. I would argue that this is a step further than that, however, with the actual validation itself needing special case logic once the correct definition has been found. If this approach is used, it should be noted in [Principles].

The `Value` field within a `FieldConstraint` in a message definition currently defines a literal value that the field must hold. It is this constraint that is used to restrict the version number to a particular major and minor version. Perhaps the message definition restriction on a field value should reflect the `Constraint` field in a microformat `Type` definition, to further restrict the values a type can take, rather than specifying a literal value. An additional field to supply a default value could then be added, with the `Value` field having its original meaning for fields where a single literal value is always required. This would add flexibility to all message definitions which want to restrict values from the base microformat definition.

Under this scheme, the definition `type/version` field for the `MsgDistMf` microformat within a message definition might change from:

```
<FieldConstraint>
    <Name>type/version</Name>
    <Multiplicity>noChange</Multiplicity>
    <Value>1.0</Value>
</FieldConstraint>
```

to

```
<FieldConstraint>
    <Name>type/version</Name>
    <Multiplicity>noChange</Multiplicity>
    <Constraint pattern="1.[0-9]{1,3}"/>
    <DefaultValue>1.0</DefaultValue>
</FieldConstraint>
```

Alternatively, the status of a the `Value` field could be downgraded from a constraint to simply specifying the default value that should be placed in the field, with no expectation that a validator should check the value beyond the underlying validation required by the constraints required by the microformat definition.

4. Applications

Requirements for applications that send and receive messages are much the same as for validators, albeit with a possibly smaller set of applicable messages. Indeed, it is likely that an application will want to validate the messages that it receives from the network before processing them, and also check that the messages that it is sending are valid. If an application is only interested in sending and receiving messages for one fixed set of messages, e.g. A single IDD version which references a specific message set, then only the definitions used in that set need be available. However, if an application wishes to communicate with other applications which might be on a different version of that IDD then it will need definitions of all message and microformat versions used by any version of the IDD with which it wishes to communicate.

5. Code Generators

Validators and applications typically do not work with the raw definitions themselves, but utilise code generated from those definitions to do message validation, encoding and decoding. If a code generator is generating code for a specific application or validator, then definitions for all messages and microformats used by that application will need to be available to generate code from. However, code generators are often used to generate libraries of code for use by multiple applications. In this case, definitions of all versions of messages that might someday be used by an application need to be available to the code generator, along with the versions of all microformat definitions used by those messages. Moreover, if the microformat libraries are generated separately from, and then used by the message libraries, all versions of all microformats that might someday be used by a message definition will need to be available.

5.1. Generated Code Compatibility

The constraints regarding changes between minor version mentioned in Section 1.1.1, “Microformat Version Compatibility” and Section 1.2.1, “Message Version Compatibility” above guarantee that messages created under one definition will be compatible with a message created with a definition with a matching major version number.

However, there are no such guarantees that code generated with one minor version will be

compatible with code generated for a different minor version. For example, take the following Microformat definitions, generated pseudo code, and application code using the generated code.

Version 1.0

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<MicroFormat root="GreetMf" version="1.0">
    <Entity name="GreetMf">
        <Primitive type="string" xmlType="element" name="greeting" id="1"
multiplicity="required"/>
        <Primitive type="string" xmlType="element" name="field2" id="2" multiplicity=
"optional"/>
    </Entity>
</MicroFormat>
```

Version 1.0 Generated Code

```
class GreetMf {
  greeting : string;
  name : optional string;
}
```

Version 1.0 Application Code

```
greet : GreetMf;
greet.greeting = 'hello';
greet.name = 'world';
```

If we now update the definition of `GreetMf` to remove the `name` field, which would be a minor version compatible change under the rules above, we would get

Version 1.1

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<MicroFormat root="GreetMf" version="1.1">
    <Entity name="GreetMf">
        <Primitive type="string" xmlType="element" name="greeting" id="1"
multiplicity="required"/>
    </Entity>
</MicroFormat>
```

Version 1.1

```
class GreetMf {
  greeting : string;
}
```

The application code from version 1.0 is now broken, as it is trying to access a field that no longer

exists. It is impossible to maintain both forwards and backwards compatibility of generated source code in any sane target language and generation scheme in the face of field additions or removals, even where those fields are optional. A similar argument can be made for addition and removal of microformats in message definitions.

If however, we disallow removal of fields from one minor version to the next, we can maintain backwards compatibility of generated code. This would allow an application written against a library of code generated from one minor version to upgrade the minor version of the library without change.

An alternative scheme would require the application to have access to code generated against all minor versions of the definitions, and to be able to select an explicit version of the code for use. This would need all version definitions to be available at code generation time, and code generator support for multiple concurrent versions.

6. Distribution of Definitions

6.1. Current Position

There are three main methods of distribution for definitions:

- Archive files containing versioned sets of definition files.
- Git repositories containing sets of definition files, with versioned sets available under version tags, and 'working' versions available as `LATEST` on branches.
- Code generated from the definitions in a particular target language.

An application or code generator might use either of these distribution methods depending on its needs. For example, an application that needs a fixed set of messages might use archive files, whereas a code generator generating libraries of code for use by multiple other applications in a CI/CD environment might use the git repositories.

6.1.1. Archive Files

The archive files currently produced provide sets of either microformats or messages from a particular version of a definition domain. For example, cs-domain-x.y.z.zip might contain microformat definitions from version x.y.z of the Common Services domain, and cs-messages-a.b.c.zip might contain message definitions from version a.b.c of the Common Services messages. Each of these will only have one version of each definition, which means that dependencies between packages of definitions must be very carefully managed such that the messages package only requires microformats at the exact version number contained within the microformat package. Finding a consistent set of packages rapidly becomes unmanageable, and likely impossible, with complex hierarchies of packages, especially when common collection definitions are used across multiple packages, and different packages are versioned independently.

If an application or validator was required to work against multiple different package versions, it would need to have archives of all those different versions available, along with all dependency versions used. Consider also the situation where two messages in a package use two different

versions of a microformat, or where a collection used by a message uses one version, and the message itself uses another version. In these cases, choosing a single package version to use would not be able to provide all the definitions required, and multiple versions of the packages would be required at one. Two different versions of the same archive are likely to have many repeated definitions, as it is likely that only a subset of definitions would change between versions.

6.1.2. Git Repositories

There are currently a large number of different git repositories containing either microformat or message/collection definitions, corresponding to the archive packages above. Tagged commits within these repositories correspond to versions of these packages. An application or code generator wishing to use specific versions must check out the tags corresponding to the versions they wish to use, with all the associated dependency management problems mentioned above.

For a code generator in a CI/CD environment, generating packages for use by multiple applications it is almost impossible to find a consistent set of versions, as it will likely be working either from the latest tagged version of each repository, or from a `LATEST` commit on a branch. As soon as a microformat version is changed, a message using an older version of that microformat would not be able to find a valid definition.

6.1.3. Generated Code

Many applications make use of generated code to form messages, and these are typically distributed as versioned packages. An application only using a single set of message versions would form the same dependency tree as mentioned above to find a consistent set of definitions and find the language packages generated from those definition versions.

Most development and runtime environments will only allow one version of a package dependency to be loaded at once. Certainly Python and Java are constrained in this way without recourse to low-level jiggery-pokery with class loaders.

This means that an application requiring multiple versions of a package would be impossible with the current way of packaging up definitions.

7. Summary

7.1. Problems

- Versioning of External and Supertype Microformat references
- `type/version` field compatibility
- Dependency Hell
- Generated code backwards compatibility
- Applications required to support multiple IDDs with conflicting versions
- Applications required to support multiple versions of the same IDD.

7.2. Solutions

Extending the DSL as detailed in [SPARTAN SHIELD Proposal] would solve the versioning of microformat references. However, see below for a discussion on whether major and minor or just major version needs to be specified.

- Add optional `version` attribute to `External` and `Supertype` elements, with default of `1.0` if not present.

Two options are given in Section 3.1, "Facets" regarding facet compatibility.

- Replace `Value` with `Constraint` and `DefaultValue` fields.
- Treat `Value` as a default value and do not validate against it.

I see three slightly different solutions to the remaining problems, all of which require keeping some older definitions available.

7.2.1. Option 1: Maintain all previous major and minor versions.

- Definitions available for all major and minor versions of messages and microformats.
- All references point to a major and minor version, as at present. The new External/Supertype references also specify minor version.
- Code generators generate code for all available versions.
- Application code explicitly selects which major and minor version it requires. This could be done on an individual message level, or via an IDD version which in turn specifies its message versions, as per [SPARTAN SHIELD Proposal].
- Validation code can validate against the exact version specified in the realized message, or against the version selected by the application as circumstances require.
- Old versions only removed once they are no longer used by any definition in any IDD.

Pros

- Any message can be fully validated from definitions or by generated code.
- No restriction on mixing versions of microformats and messages across or within IDDs.
- Stable generated code for applications across version changes.

Cons

- Explosion of definition versions
- Explosion of generated code
- Hard to keep definitions up to date with references to the latest versions - every minor version bump requires all references to be updated.

7.2.2. Option 2: Maintain the latest minor version for all major versions.

- Definitions available for all major and latest minor versions of messages and microformats.
- All references change to only reference a major version.
- (Optional) Messages on the wire only specify their major version - change to `MsgDistMf`.
- Code generators generate code for each major version, based on the latest available minor version.
- Application code explicitly selects the major version it requires, and receives the latest available minor version.
- Validation done against the latest available minor version.
- Old minor versions removed when superseded.
- Old major versions only remove once they are no longer used by any definition in any IDD.

Pros

- Minimal set of old versions maintained
- Minimal generated code
- No restriction on mixing versions of microformats and messages across or within IDDs.
- Minor version in message and microformat references can be removed.
- Less version maintenance overhead - referencing definitions only need updating when the major version changes.

Cons

- Backwards compatibility of generated can only be maintained if field or microformat removal causes major version increment.
- No forward compatibility of generated code.
- Validation is only against the latest minor version, which may not be sufficient for SPARTAN SHIELD.

7.2.3. Option 3: Best and worst of both worlds

This is basically Option 2, but with all definitions remaining available for SPARTAN SHIELD validation.

- Definitions available for all major and minor versions of messages and microformats.
- All references change to only reference a major version.
- Messages on the wire continue to specify major and minor version.
- Code generators generate code for each major version, based on the latest available minor version.
- Application code explicitly selects the major version it requires, and receives the latest available minor version.

- Application validation done against the latest available minor version.
- SPARTAN SHIELD validation done against exact version specified in the realized message.

Pros

- Any message can be fully validated by a validator using definitions e.g. SPARTAN SHIELD
- Minimal generated code for general applications.
- No restriction on mixing versions of microformats and messages across or within IDDs.
- Minor version in message and microformat references can be removed.
- Less version maintenance overhead - referencing definitions only need updating when the major version changes.

Cons

- Backwards compatibility of generated code can only be maintained if field or microformat removal causes major version increment.
- No forward compatibility of generated code.

7.2.4. Old Version Availability

All of the above options require that definitions are available for older versions of microformats and messages. There are a number of ways this could be done

- Add the version number as part of the filename for all definitions. If Option 2 above is selected, this only need be the major version.
- Archive older versions into a different directory and add a version number to the filename.
- Modify the DSL to allow multiple versions to be contained within the same definition file.
- Modify the DSL to allow version ranges to be specified on individual fields or microformats specifying which versions they are valid for.

I would suggest that the simplest option is the first of these.